Kubernetes on RHEL 9–A step-by-step guide

medium.com/weeklycloud/kubernetes-installation-on-rhel-9-d5629f2fa4f9

Jerome Decinco November 11, 2024

In this article, I will guide you through the step-by-step process of installing Kubernetes on RHEL 9, covering essential setup requirements, configuration tips, and troubleshooting common issues to ensure a smooth deployment.

> Published on weeklycloud. Join our newsletter weeklycloud.info to get weekly cloud insights ☁️

What is kubernetes?

Kubernetes (K8s) is a powerful open-source platform for automating deployment, scaling, and management of containerized applications, originally developed by Google and now managed by the Cloud Native Computing Foundation. Inspired by Google's Borg, Kubernetes organizes machines into a cluster that runs workloads in containers, and it supports multiple container runtimes like containerd and CRI-O. Its flexibility has led to widespread adoption in data centers and cloud platforms.

Prerequisites

- **Admin rights**: Sudo user
- **Control Plane Node**: Minimum 2 CPU cores and 2GB of RAM (4GB+ is recommended).
- **Worker Nodes**: At least 1 CPU core and 1GB of RAM (2GB+ is recommended).
- **Operating System:** A compatible Linux distribution (e.g., Rocky Linux 9, RHEL 9, CentOS 9, Ubuntu 20.04+, Debian 10+, etc.).

1# Step: set up hostname and hosts configuration

Setting up a unique hostname and configuring the /etc/hosts file are vital for Kubernetes cluster setup. Unique identifiers for each node (control plane or worker) ensure clear differentiation during scheduling and operations, like naming the control plane node k8s-master01. Proper network communication is essential, as components rely on effective DNS resolution, which is facilitated by /etc/hosts. Descriptive hostnames simplify management, particularly in larger clusters. Standardized naming promotes consistent configurations, reducing errors. To configure, assign unique hostnames and populate /etc/hosts with corresponding IP addresses, establishing a robust foundation for smooth communication and scalability in the cluster.

The control plane node should have a unique name. (e.g., k8s-master01)

```
$ sudo hostnamectl set-hostname "k8s-master01" && exec bash
```

Workers node should have a numerical order (e.g., k8s-worker01..02..03)

```
$ sudo hostnamectl set-hostname "k8s-worker01" && exec bash
```

Add the hosts on each node in /etc/hosts

```
{ip_address}   k8s-master01
{ip_address}   k8s-worker01
{ip_address}   k8s-worker02
{ip_address}   k8s-worker03
```

2# Step: Disable Swap Space on Each Node

Disabling swap space is essential for Kubernetes setup as it ensures predictable resource management. Kubernetes depends on accurate CPU and memory reporting from nodes, and having swap can introduce unpredictable memory usage, affecting scheduling and stability. The kubelet needs precise control over memory; with swap on, it struggles to assess available memory, leading to performance degradation and potential node failures. Additionally, Kubernetes utilizes Quality of Service (QoS) classes to prioritize resource allocation among containers, but swap can interfere with this, particularly for “Guaranteed” containers. The Kubernetes team mandates disabling swap during installation, as enabling it may cause setup warnings or failures.

```
$ sudo swapoff -a
$ sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
```

3# Step: Adjust SELinux and Firewall Rules

SELinux, a security module in Linux, is often set to permissive mode in Kubernetes clusters for compatibility, ease of troubleshooting, and to avoid operational issues. Permissive mode allows container runtimes and components to operate without restrictions, logs security violations instead of blocking actions, and prevents SELinux from interfering with network and file access required by Kubernetes pods and services.

```
$ sudo setenforce 0
$ sudo sed -i --follow-symlinks 's/SELINUX=enforcing/SELINUX=permissive/g'
/etc/sysconfig/selinux
```

For Kubernetes components to communicate effectively across nodes, certain ports must be opened in the firewall. These ports enable essential Kubernetes communication and control

functions:

- **6443/tcp**: Kubernetes API server
- **2379–2380/tcp**: etcd server (used for storing cluster data)
- **10250–10252/tcp**: kubelet API and control plane services
- **10257–10259/tcp**: Scheduler and controller manager
- **179/tcp**: BGP (for networking plugins, if used)
- **4789/udp**: VXLAN (for pod networking, if using overlay networks)

Commands to Open Ports on the Control Plane Node

```
$ sudo firewall-cmd --permanent --add-port=
{6443,2379,2380,10250,10251,10252,10257,10259,179}/tcp
$ sudo firewall-cmd --permanent --add-port=4789/udp
$ sudo firewall-cmd --reload
```

These ports facilitate node-to-node communication and pod access:

- **10250/tcp**: kubelet API on worker nodes
- **30000–32767/tcp**: NodePort range for services exposed to external access
- **179/tcp**: BGP (if using)
- **4789/udp**: VXLAN (for overlay network communication)

Commands to Open Ports on Worker Nodes

```
$ sudo firewall-cmd --permanent --add-port={179,10250,30000-32767}/tcp
$ sudo firewall-cmd --permanent --add-port=4789/udp
$ sudo firewall-cmd --reload
```

4# Step: Add Kernel Modules and Parameters

Adding kernel modules and parameters is crucial for Kubernetes to manage network traffic and container communication across nodes. The overlay module facilitates layered filesystems for efficient storage in containers, while br_netfilter allows network filtering between bridged interfaces. These modules ensure proper inter-pod networking and adherence to Kubernetes' network policies, preventing performance issues and enhancing functionality.

```
$ sudo echo -e "overlay\nbr_netfilter" | sudo tee /etc/modules-load.d/containerd.conf
&& sudo modprobe overlay && sudo modprobe br_netfilter
```

Kubernetes requires specific kernel parameters for effective network traffic management:

1. **net.bridge.bridge-nf-call-iptables = 1** enables bridged traffic to pass through iptables, crucial for routing between nodes and pods.
2. **net.ipv4.ip_forward = 1** allows IP forwarding for pod-to-pod communication across node interfaces.
3. **net.bridge.bridge-nf-call-ip6tables = 1** manages IPv6 traffic on bridged interfaces through ip6tables, important for IPv6 networking environments.

```
$ sudo echo -e "net.bridge.bridge-nf-call-iptables = 1\nnet.ipv4.ip_forward =
1\nnet.bridge.bridge-nf-call-ip6tables = 1" | sudo tee -a /etc/sysctl.d/k8s.conf
```

Then add these parameters by running these command:

```
$ sudo sysctl --system
```

5# Step: Install Containerd Runtime

Kubernetes requires a container runtime, and one of the most popular choices is containerd. But It is not available in the default package repositories of Rocky Linux or AlmaLinux, so add the following docker repo on all the nodes.

```
$ sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-
ce.repo
```

Now, run following dnf command to install containerd on all the nodes.

```
$ sudo dnf install containerd.io -y
```

Configure containerd so that it will use systemdcgroup, execute the following commands on each node.

```
$ containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1
$ sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g'
/etc/containerd/config.toml
```

or

```
$ sudo containerd config default | sudo tee /etc/containerd/config.toml | sudo sed -i
's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
```

Restart and enable containerd service using beneath commands,

```
$ sudo systemctl restart containerd
$ sudo systemctl enable containerd
```

Verify containerd service status, run

```
$ sudo systemctl status containerd
```

6# Step: Install Kubernetes tools

Kubernetes tools like Kubeadm, kubectl and kubelet are not available in the default package repositories of Rocky Linux 9 or AlmaLinux 9. So, to install these tools, add the following repository on all the nodes.

```
$ cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
```

or

```
$ sudo echo -e "
[kubernetes]\nname=Kubernetes\nbaseurl=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/\ne
nabled=1\ngpgcheck=1\ngpgkey=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/repodata/repo
md.xml.key\nexclude=kubelet kubeadm kubectl cri-tools kubernetes-cni" | sudo tee
/etc/yum.repos.d/kubernetes.repo > /dev/null
```

Note: At time of writing this post, Kubernetes 1.28 version was available, that's why I have mentioned v1.28 while adding the repo.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

Next, install Kubernetes tools by running following dnf command,

```
$ sudo yum install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
```

After installing Kubernetes tools, start the kubelet service on each node.

```
$ sudo systemctl enable --now kubelet
```

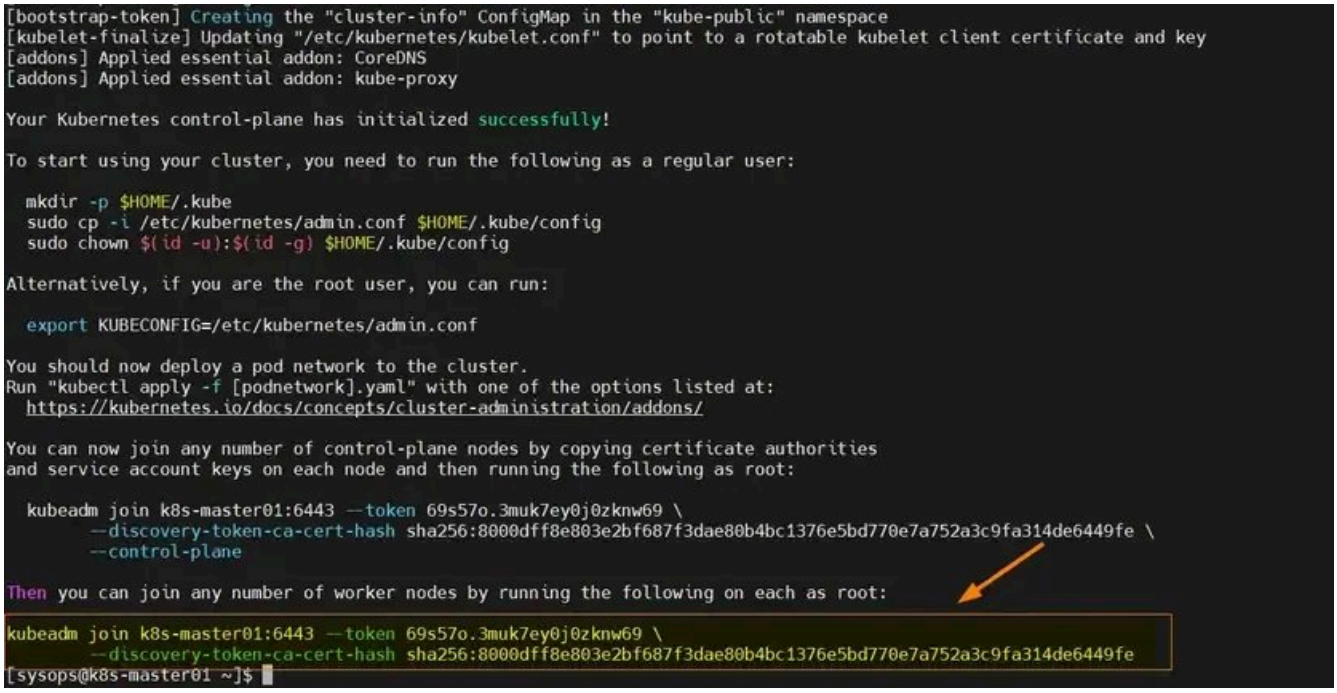
7# Step: Install Kubernetes Cluster

Now, we are all set to install Kubernetes cluster. Run beneath Kubeadm command to initialize the Kubernetes cluster from the master node.

```
$ sudo kubeadm init --control-plane-endpoint=k8s-master01
```

Once above command is executed successfully, we will get following output,

Press enter or click to view image in full size



From the output above make a note of the command which will be executed on the worker nodes to join the Kubernetes cluster.

To start interacting with Kubernetes cluster, run the following commands on the master node.

```
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

or

```
$ mkdir -p $HOME/.kube && sudo bash -c "cp -i /etc/kubernetes/admin.conf
$HOME/.kube/config && chown $(id -u):$(id -g) $HOME/.kube/config"
```

Next, join the worker nodes to the cluster, run following Kubeadm command from the worker nodes.

```
$ kubeadm join k8s-master01:6443 --token 69s57o.3muk7ey0j0zknw69 \
  --discovery-token-ca-cert-hash sha256:8000dff8e803e2bf687f3dae80b4bc1376e5bd770e7a75
```